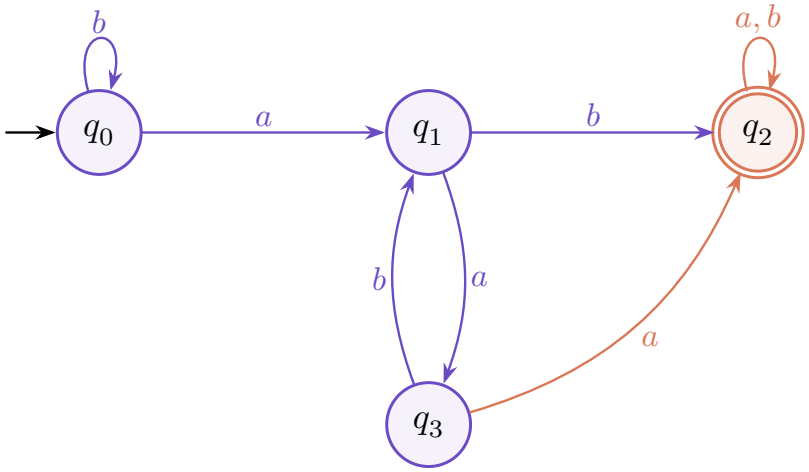
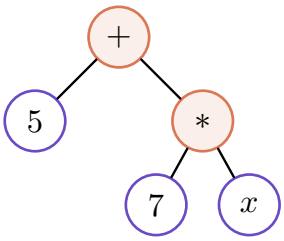
Информатика




```
S ::= E $
E ::= T { + T }
T ::= F { * F }
F ::= num | ( E )
```



Содержание

1 Цепочки и языки 3

2 Грамматики 4

2.1 Определение грамматики и пример вывода . . . 4
2.2 Выводимость и сентенциальные формы . . . 4
2.3 Эквивалентность грамматик . . . 4

3 Классификация Хомского 5

3.1 Типы грамматик . . . 5
3.2 Тип языка и обозначения . . . 5
3.3 Как определить тип грамматики . . . 5

4 Структура вывода в КС-грамматиках 6

4.1 Левый и правый вывод . . . 6
4.2 Дерево вывода. Неоднозначность . . . 6
4.3 Недостижимые и бесплодные символы . . . 7

5 Преобразование грамматик 8

5.1 Очистка грамматики . . . 8
5.2 Избавление от ε-правил . . . 9

6 Компиляция на практике 9

6.1 Этапы компиляции . . . 9
6.2 Подходы к компиляции . . . 10
6.3 Распознающие формализмы . . . 10

7 Регулярные языки 11

7.1 НКА и ДКА . . . 11
7.2 Автоматы и линейные грамматики . . . 12
7.3 Построение ДКА . . . 13
7.4 Операции над языками и регулярные выражения . . . 15
7.5 Регулярные выражения на практике (PCRE) . . . 16
7.6 Группировки . . . 17
7.7 Лексический анализ на практике . . . 18
7.8 Форма Бэкуса–Наура (BNF) . . . 18

8 Синтаксический анализ 19

8.1 Пример и классификация алгоритмов . . . 19
8.2 Рекурсивный спуск . . . 20
8.3 Применимость рекурсивного спуска . . . 22

1 Цепочки и языки

Первичное понятие: символ.

Определение. **Алфавит** — конечный набор символов. Обычно обозначается Σ (V).

Определение. **Цепочка** символов в алфавите Σ — любая конечная последовательность в алфавите Σ.

Определение. **Пустая цепочка** — цепочка, не содержащая символов. Обозначается ε.

Обозначения: буквы — $a,\ b,\ c,\ 0,\ 1,\ 2$; цепочки — $\alpha,\ \beta,\ \gamma,\ \xi$.

Определение. **Конкатенация** цепочек α и β — это $\alpha\beta$: цепочка, в которой сначала записаны символы α, потом β.

Определение. **Реверс** цепочки α: $\alpha^R = a_n \cdots a_1$, если $\alpha = a_1 \ldots a_n$.

Определение. **Степень** цепочки α: $\alpha^n = \underbrace{\alpha\alpha \ldots \alpha}_{n}$.

Определение. $|\alpha|$ — **длина** цепочки (количество символов); $|\alpha|_a$ — количество символов a в α.

Определение. Σ^* — множество всех цепочек, состоящих из символов Σ. Здесь * — **звёздочка Клини**. Например, при $\Sigma = \{0, 1\}$:

$$\Sigma^* = \{\varepsilon,\ 0,\ 1,\ 00,\ 01,\ 10,\ 11,\ 000, \ldots\}$$

Определение. Σ^+ — множество всех **непустых** цепочек из символов Σ. Здесь $^+$ — **плюс Клини**, и $\Sigma^+ = \Sigma^* \setminus \{\varepsilon\}$.

Определение. **Язык** L над алфавитом Σ: $L \subseteq \Sigma^*$.

Определение. **Декартово произведение**: $A \times B = \{(a, b) \mid a \in A,\ b \in B\}$.

2 Грамматики

2.1 Определение грамматики и пример вывода

Определение. **Грамматика** G — это четвёрка объектов $G = \langle T, N, P, S \rangle$, где

- T — множество **терминалов** (*символы алфавита*);
- N — множество **нетерминалов** (*промежуточные символы*), причём $T \cap N = \emptyset$;
- P — множество **правил вывода**: $\alpha \to \beta \in P$, где $\alpha \in (T \cup N)^+$ содержит нетерминалы, а $\beta \in (T \cup N)^*$;
- S — **стартовый символ**, $S \in N$.

Обозначения. Нетерминалы: $A,\ B,\ C$.

Пример. $G:\ T = \{0, 1\},\ N = \{S, A\}$,

$$P: \begin{cases} S \to 0A1 \\ 0A \to 00A1 \\ A \to \varepsilon \end{cases}$$

Цепочка вывода:

$$S \Rightarrow 0A1 \Rightarrow \overset{\xi_1}{\hat{0}}\ \overbrace{0A1}^{\gamma}\ \overset{\xi_2}{\hat{1}} \Rightarrow 00\overbrace{0A1}^{\delta}11 \Rightarrow \dots$$

что даёт последовательно $01, 0011, 0^3 1^3$. Значит, $L = \{0^n 1^n \mid n \geq 1\}$ — язык, порождаемый грамматикой G. Эквивалентная грамматика: $P':\ S \to 0S1 \mid 01$.

2.2 Выводимость и сентенциальные формы

Определение. $\beta \in (T \cup N)^*$ **непосредственно выводима** из $\alpha \in (T \cup N)^+$ в грамматике G (обозначение $\alpha \Rightarrow_G \beta$), если $\alpha = \xi_1 \gamma \xi_2$, $\beta = \xi_1 \delta \xi_2$ и правило $\gamma \to \delta \in P$.

Определение. $\beta \in (T \cup N)^*$ **выводима** из $\alpha \in (T \cup N)^+$ в G (обозначение $\alpha \Rightarrow^*_G \beta$), если существуют $\gamma_1, \dots, \gamma_n$ такие, что $\alpha \Rightarrow \gamma_1 \Rightarrow \dots \Rightarrow \gamma_n \Rightarrow \beta$.

Определение. Язык, **порождаемый грамматикой** G: $L(G) = \{\alpha \in T^* \mid S \Rightarrow^*_G \alpha\}$.

Определение. **Сентенциальная форма** — цепочка $\alpha \in (T \cup N)^*$, выводимая из S.

2.3 Эквивалентность грамматик

Определение. G_1 и G_2 называются **эквивалентными**, если $L(G_1) = L(G_2)$.

Определение. G_1 и G_2 называются **почти эквивалентными**, если $L(G_1) \cup \{\varepsilon\} = L(G_2) \cup \{\varepsilon\}$.

3 Классификация Хомского

3.1 Типы грамматик

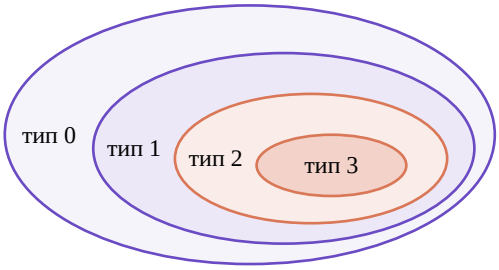
- **Тип 0.** Все возможные.
- **Тип 1.**
  - **Неукорачивающие.** Правила вывода не укорачивают слово: $\alpha \to \beta$, $|\alpha| \le |\beta|$.
  - **Контекстно-зависимые.** $\alpha A \beta \to \alpha \gamma \beta$, $|\gamma| > 0$.
- **Тип 2. Контекстно-свободные** (КС). Все правила вывода имеют вид $A \to \gamma$.
- **Тип 3.**
  - **Праволинейная**: $A \to aB$ и $A \to a$, где $A, B \in N$, $a \in T$.
  - **Леволинейная**: $A \to Ba$ и $A \to a$.

> **Замечание.** В неукорачивающих грамматиках может быть правило $S \to \varepsilon$, если S не встречается в правых частях правил.

3.2 Тип языка и обозначения

> **Определение.** **Типом языка** называется максимальный тип грамматики, задающей язык.

- Тип 0 — формальные языки, FL;
- Тип 1 — контекстно-зависимые языки, CDL;
- Тип 2 — контекстно-свободные языки, CFL;
- Тип 3 — регулярные языки, REG.



3.3 Как определить тип грамматики

1) Только один нетерминал слева — тип 2 или 3, иначе тип 0 или 1.
2) Все правила односторонние (нетерминал с одной стороны) — тип 3, иначе тип 2.

> **Почему $a^n b^n$ контекстно-свободный, но не регулярный.** Нам нужно уметь считать, сколько a мы поставили. Если ставить сначала a, потом b по n раз, число нетерминалов не будет конечным — подходит только изменение строки из середины.

4 Структура вывода в КС-грамматиках

4.1 Левый и правый вывод

Определение. **Левый** и **правый вывод** — вывод, в котором на каждом шаге раскрывается соответственно самый левый или самый правый нетерминал.

Для грамматики $S \to SS \mid a \mid b$ построим вывод слова $w = aaba$.

Левый вывод (раскрываем самый левый нетерминал):

$$\underline{S} \to \underline{S}S \to \underline{S}SS \to \underline{S}SSS \to a\underline{S}SS \to aa\underline{S}S \to aab\underline{S} \to aaba$$

Правый вывод (раскрываем самый правый нетерминал):

$$\underline{S} \to S\underline{S} \to SS\underline{S} \to SSS\underline{S} \to SS\underline{S}a \to S\underline{S}ba \to \underline{S}aba \to aaba$$

4.2 Дерево вывода. Неоднозначность

Определение. **Дерево вывода** — дерево, узлами которого являются нетерминалы, а листьями — терминалы. Дерево вывода не единственно.

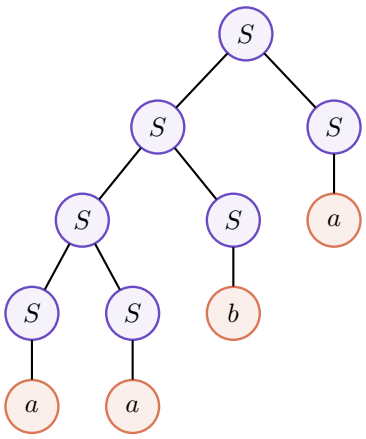


Рис. 1. Дерево вывода слова $aaba$ в грамматике $S \to SS \mid a \mid b$

Определение. Грамматика называется **неоднозначной**, если у неё существует несколько деревьев вывода для некоторой цепочки. В ином случае грамматика называется **однозначной**.

4.3 Недостижимые и бесплодные символы

Определение. **Недостижимые символы** — нетерминалы, которые не выводятся из S.

Определение. **Бесплодные символы** — нетерминалы, из которых не выводятся терминальные цепочки.

Пример.

$$S \to aAB \mid E$$
$$A \to dDA$$
$$B \to bE \mid f$$
$$C \to cAB \mid dSD \mid a$$
$$D \to eA$$
$$E \to fA \mid g$$

Недостижимые: C. Бесплодные: $A,\ D$.

5 Преобразование грамматик

5.1 Очистка грамматики

Поиск.

1) **Недостижимые:**
   a) строим $K_0 = \{S\}$;
   b) строим $K_i = K_{i-1} \cup \{B \mid \exists A \to \xi_1 B \xi_2,\ A \in K_{i-1}\}$;
   c) конец, когда $K_i = K_{i-1}$.
2) **Бесплодные:**
   a) строим $K_0 = \Sigma$ (то есть T);
   b) строим $K_i = K_{i-1} \cup \{A \mid \exists A \to \gamma,\ \gamma \in K_{i-1}^*\}$;
   c) конец, когда $K_i = K_{i-1}$.

Удаление.

1) Удаляем бесплодные.
2) Удаляем недостижимые.

Пример а).

$$\begin{aligned} S &\to aABS \mid bCACd \\ A &\to bAB \mid cSA \mid cCC \\ B &\to bAB \mid cSB \\ C &\to cS \mid c \end{aligned}$$

Плодовитые:

$$\begin{aligned} K_0 &= \{a, b, c, d\} \\ K_1 &= \{a, b, c, d, C\} \\ K_2 &= \{a, b, c, d, C, A\} \\ K_3 &= \{a, b, c, d, C, A, S\} \\ K_4 &= K_3 \Rightarrow \text{конец} \end{aligned}$$

$\Rightarrow$ бесплодные: B. Тогда

$$P: \begin{aligned} S &\to bCACd \\ A &\to cSA \mid cCC \\ C &\to cS \mid c \end{aligned}$$

Достижимые: $K_0 = \{S\}$, $K_1 = \{S, C, A\}$, $K_2 = K_1 \Rightarrow$ конец $\Rightarrow$ недостижимых нет.

Пример б).

$$\begin{aligned} S &\to ab \mid aA \\ A &\to bAB \mid aA \\ B &\to aB \mid b \end{aligned}$$

Плодовитые: $K_0 = \{a, b\}$, $K_1 = \{a, b, S, B\}$, $K_2 = K_1 \Rightarrow$ конец $\Rightarrow$ бесплодные: A. Тогда $P: S \to ab,\ B \to aB \mid b$.

Достижимые: $K_0 = \{S\}$, $K_1 = K_0 \Rightarrow$ конец $\Rightarrow$ недостижимые: B, и остаётся $P: S \to ab$.

5.2 Избавление от ε-правил

$$\begin{array}{l} A \to aB \mid \varepsilon \\ C \to DABA \end{array} \quad \Longrightarrow \quad \begin{array}{l} A \to aB \\ C \to DABA \mid DB \mid DAB \mid DBA \end{array}$$

Алгоритм.

1) Строим N' — множество **«схлопывающихся»** нетерминалов: $N' = \{A \in N \mid A \to^* \varepsilon\}$.
2) Убираем правила вида $A \to \varepsilon$. Если $\varepsilon \in L$, то вводим новый стартовый символ S' с правилом $S' \to S \mid \varepsilon$.
3) Для всех правил вида $B \to \alpha_0 A_1 \alpha_1 A_2 \alpha_2 \dots A_n \alpha_n$, где $A_i \in N'$ и $\alpha_i \in (T \cup (N \setminus N'))^*$, добавляем правила со всеми возможными выбрасываниями A_i:

$$\begin{array}{l} B \to \alpha_0 \alpha_1 \dots \alpha_n \\ B \to \alpha_0 \alpha_1 \dots A_n \alpha_n \\ \dots \\ B \to \alpha_0 A_1 \alpha_1 A_2 \dots A_n \alpha_n \end{array}$$

4) Удалить бесплодные и недостижимые.

6 Компиляция на практике

Как это используется на практике. Для компиляторов используют регулярные (тип 3) и контекстно-свободные (тип 2) языки.

6.1 Этапы компиляции

1) **Лексический анализ.** Текст → поток токенов:

`if(a<5)` → `if` (служеб. слово) `(` `a` (имя) `<` (опер.) `5` (число) `)`

2) **Синтаксический анализ.** Поток токенов → синтаксическое дерево (рис. 2).
3) **Семантический анализ.** Проверка на смысловую корректность.
4) **Генерация кода.**

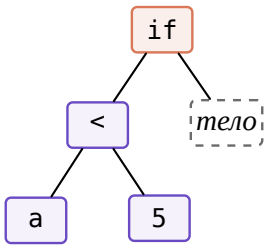


Рис. 2. Синтаксическое дерево для `if(a<5)`

6.2 Подходы к компиляции

1) **Традиционная компиляция.** `file.cpp` → `file.exe` (можно сразу запустить). Быстрые компиляция и работа.
2) **JIT-компиляция** (JIT — Just In Time). `file.cs` → IL (промежуточный язык), компиляция в машинные коды при запуске. Медленная компиляция, быстрая работа, безопасность.
3) **Интерпретация.** `file.py`. Медленная работа, кроссплатформенность, безопасность.

6.3 Распознающие формализмы

Грамматика — **порождающий** формализм. Ему двойственны **распознающие**:

Тип языка	Распознающий формализм
Тип 3	Конечные автоматы
Тип 2	МП-автоматы (автоматы с магазинной памятью)
Тип 1	ЛОА (линейно ограниченный автомат)
Тип 0	Машина Тьюринга

Таблица 1. Порождающие и распознающие формализмы

7 Регулярные языки

7.1 НКА и ДКА

Определение. Недетерминированный конечный автомат (НКА) — пятёрка объектов $\langle Q, \Sigma, \delta, q_0, F \rangle$:

- Q — конечное множество состояний;
- Σ — алфавит;
- $\delta : \ Q \times (\Sigma \cup \{\varepsilon\}) \to 2^Q$ (множество всех подмножеств Q) — функция перехода (переход не однозначен);
- $q_0 \in Q$ — начальное состояние;
- $F \subseteq Q$ — множество конечных состояний.

Определение. Детерминированный конечный автомат (ДКА) — пятёрка объектов $\langle Q, \Sigma, \delta, q_0, F \rangle$:

- Q — конечное множество состояний;
- Σ — алфавит;
- $\delta : \ Q \times \Sigma \to Q \cup \{\emptyset\}$ — функция перехода (переход всегда однозначен);
- $q_0 \in Q$ — начальное состояние;
- $F \subseteq Q$ — множество конечных состояний.

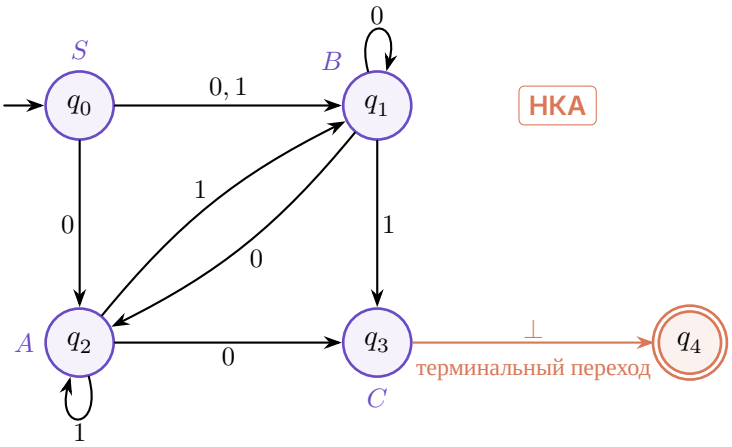


Рис. 3. Автомат к примеру 1

Праволинейная грамматика:

$$S \to 0A \mid 0B \mid 1B$$
$$A \to 1A \mid 1B \mid 0C$$
$$B \to 0A \mid 0B \mid 1C$$
$$C \to \perp$$

Леволинейная грамматика:

$$D \to C \perp$$
$$C \to B1 \mid A0$$
$$B \to B0 \mid A1 \mid 0 \mid 1$$
$$A \to A1 \mid B0 \mid 0$$

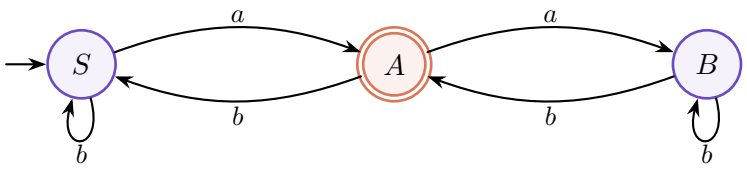


Рис. 4. Автомат к примеру 2

Праволинейная грамматика:

$$S \to aA \mid bS \mid a$$
$$A \to bS \mid aB$$
$$B \to bB \mid bA \mid b$$

Леволинейная грамматика:

$$A \to Sa \mid Bb \mid a$$
$$B \to Ba \mid Aa$$
$$S \to Sb \mid Ab \mid b$$

7.2 Автоматы и линейные грамматики

Пусть есть автомат $\langle Q, \Sigma, \gamma, q_0, F\rangle$ и грамматика $\langle T, N, S, P\rangle$. Тогда можно сопоставить:

Для праволинейных:

- $Q \to N, \quad \Sigma \to T,$
- $q_0 \to S, \quad F \to -,$
- $\gamma \to P.$

Для леволинейных:

- $Q \to N, \quad \Sigma \to T,$
- $q_0 \to -, \quad F \to S,$
- $\gamma \to P.$

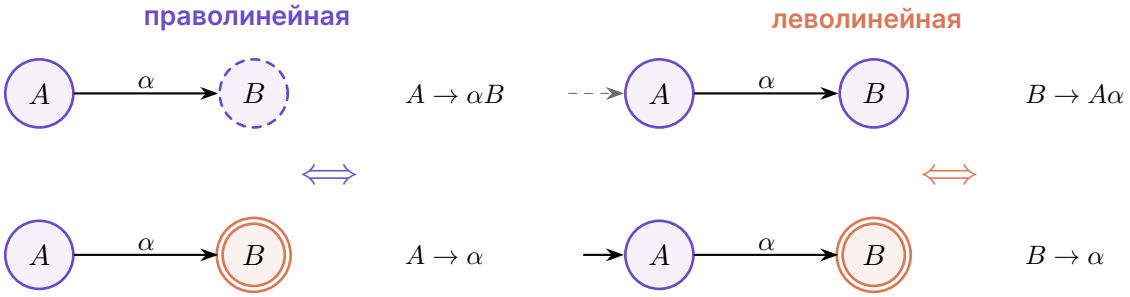
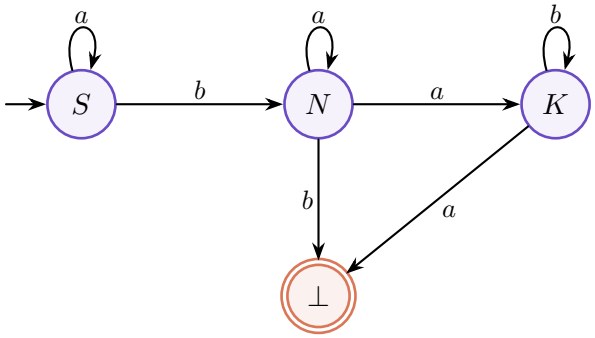


Рис. 5. Соответствие автомата и линейной грамматики (пунктирное состояние — любое)

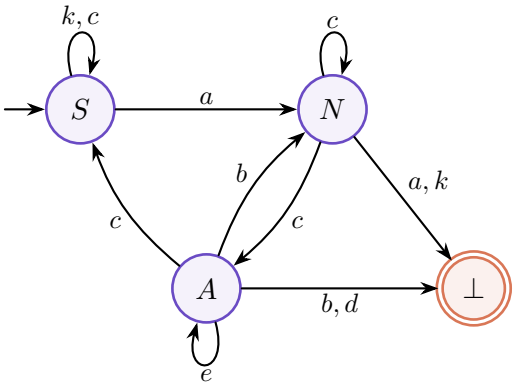
Пример 3.

$$S \to aS \mid bN$$
$$N \to aN \mid aK \mid b$$
$$K \to a \mid bK$$



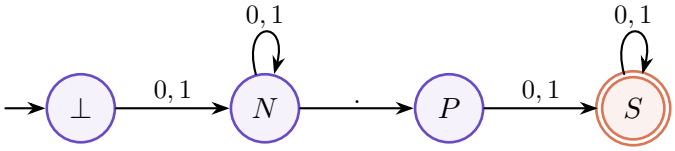
Пример 4.

$$S \to aN \mid kS \mid cS$$
$$N \to a \mid cN \mid cA \mid k$$
$$A \to b \mid bN \mid eA \mid d \mid cS$$



Пример 5 (леволинейная грамматика).

$$S \to S0 \mid S1 \mid P0 \mid P1$$
$$P \to N.$$
$$N \to 0 \mid 1 \mid N0 \mid N1$$



7.3 Построение ДКА

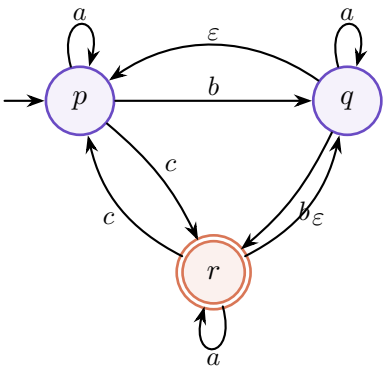
Разберём на примере 3. Состояния ДКА — множества состояний НКА:

ДКА	НКА	a	b
Q_0	S	Q_0	Q_1
Q_1	N	Q_2	Q_3
Q_2	N, K	Q_4	Q_5
Q_3	$\perp$	—	—
Q_4	$N, K, \perp$	Q_4	Q_5
Q_5	$K, \perp$	Q_3	Q_6
Q_6	K	Q_3	Q_6

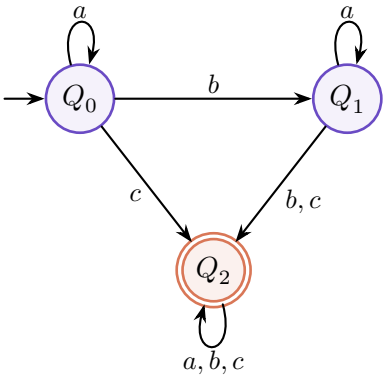
Таблица 2. Построение ДКА по НКА из примера 3

Пример 6. НКА задан таблицей переходов:

	ε	a	b	c
$\to p$	$\emptyset$	$\{p\}$	$\{q\}$	$\{r\}$
q	$\{p\}$	$\{q\}$	$\{r\}$	$\emptyset$
$^*r$	$\{q\}$	$\{r\}$	$\emptyset$	$\{p\}$



ДКА	НКА	a	b	c
$\to Q_0$	p	Q_0	Q_1	Q_2
Q_1	q, p	Q_1	Q_2	Q_2
$^*Q_2$	q, p, r	Q_2	Q_2	Q_2



Пример 7.

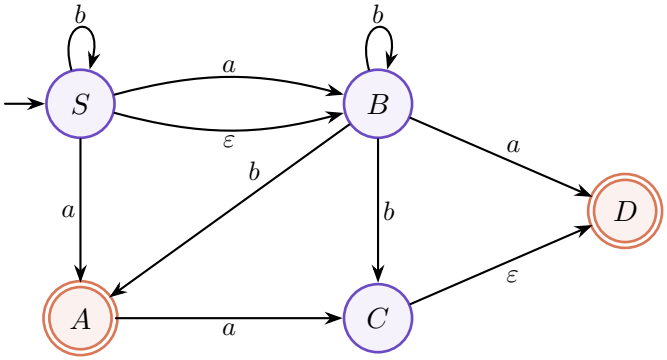
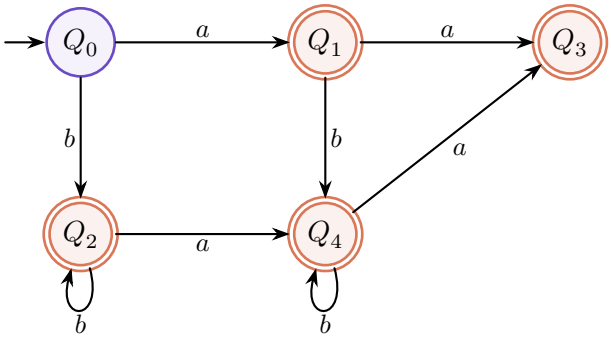


Рис. 6. НКА к примеру 7

ДКА	НКА	a	b
$\to Q_0$	SB	Q_1	Q_2
$^*Q_1$	ABD	Q_3	Q_4
$^*Q_2$	$SABCD$	Q_4	Q_2
$^*Q_3$	CD	$\emptyset$	$\emptyset$
$^*Q_4$	$ABCD$	Q_3	Q_4



Праволинейная грамматика (убрали Q_3, т. к. бесплодное):

$$Q_0 \to aQ_1 \mid bQ_2 \mid a \mid b$$
$$Q_1 \to bQ_4 \mid a \mid b$$
$$Q_2 \to aQ_4 \mid bQ_2 \mid a \mid b$$
$$Q_4 \to bQ_4 \mid a \mid b$$

Леволинейная грамматика:

$$\perp \to Q_1 \mid Q_2 \mid Q_3 \mid Q_4$$
$$Q_3 \to Q_1a \mid Q_4a$$
$$Q_4 \to Q_4b \mid Q_2a \mid Q_1b$$
$$Q_2 \to Q_2b \mid b$$
$$Q_1 \to a$$

7.4 Операции над языками и регулярные выражения

Определение. Конкатенация языков: $AB = \{\alpha\beta \mid \alpha \in A,\ \beta \in B\}$.

Пример. $A = \{a, bb\}$, $B = \{ba, cbc\} \Rightarrow AB = \{aba,\ acbc,\ bbba,\ bbcbc\}$.

Определение. Степень языка: $L^n = \underbrace{LL\ldots L}_{n}$, $L^0 = \{\varepsilon\}$.

Пример. $A = \{a, bb\}$: $A^3 = \{aaa,\ aabb,\ abba,\ abbbb,\ bbaa,\ bbabb,\ bbbba,\ bbbbbb\}$.

Определение. Замыкание Клини: $L^* = \bigcup\limits_{n=0}^{\infty} L^n$.

Пример. $A = \{a, bb\}$: $A^* = \{\varepsilon,\ a,\ bb,\ aa,\ abb,\ bba,\ bbbb, \ldots\}$.

Определение. Плюс Клини: $L^+ = \bigcup\limits_{n=1}^{\infty} L^n = LL^*$.

Определение. **Регулярное выражение** состоит из: $\varepsilon,\ \Sigma,\ (,\ ),\ ^*,\ |,\ \cdot,\ \emptyset$.

1) $\emptyset$, ε, a ($a \in \Sigma$) — регулярные выражения.
2) Если α и β — регулярные, то $\alpha|\beta$, $\alpha \cdot \beta$, α^*, (α) — регулярные.
3) Других регулярных выражений нет.

Определение. Язык, задаваемый **регулярным выражением**:

1) $L(\emptyset) = \emptyset, \quad L(\varepsilon) = \{\varepsilon\}, \quad L(a) = \{a\}$;
2) $L(\alpha \cdot \beta) = L(\alpha)L(\beta), \quad L(\alpha^*) = L(\alpha)^*, \quad L((\alpha)) = L(\alpha), \quad L(\alpha|\beta) = L(\alpha) \cup L(\beta)$.

Пример. Язык всех слов над $\{a, b\}$, заканчивающихся на abb. Регулярное выражение: $(a|b)^* \cdot a \cdot b \cdot b$.

7.5 Регулярные выражения на практике (PCRE)

Регулярное выражение — шаблон для поиска, проверки и замены текста. В библиотеках синтаксис богаче, чем в определении выше: есть обратные ссылки, которые выходят за рамки регулярных языков.

Диалекты: POSIX (BRE, ERE; `grep`, `sed`), PCRE (Perl-Compatible Regular Expressions, самый богатый синтаксис). В C++ `std::regex` использует ECMAScript, близкий к PCRE (но без lookbehind).

Функции в C++ (`<regex>`):

- `regex_match` — проверка **полного** совпадения строки;
- `regex_search` — поиск **первого** вхождения;
- `regex_replace` — замена совпадающих частей.

Результат лежит в `std::smatch` / `std::cmatch`: `res[0]` — всё совпадение, `res[1]`, `res[2]`, … — группы.

Замечание. В строке C++ обратный слеш удваивается, проще писать raw-строку: `"\\d+"` ⇒ `R"(\d+)"`.

Определение. **Метасимволы** — символы со специальным смыслом:

```
. ^ $ * + ? ( ) [ ] { } | \
```

Чтобы использовать метасимвол как обычный, его **экранируют**: `\.`, `\*`, `\\`.

Определение. **Одиночные метасимволы**:

- `.` — любой символ, кроме `\n`;
- `^` — начало строки, `$` — конец строки;
- `\b` — граница слова, `\B` — не граница слова.

Замечание. `\b` и `\w` работают только для латиницы.

Определение. **Символьные классы** (заглавная буква — инверсия):

- `\s` (`\S`) — пробельные символы;
- `\d` (`\D`) — цифры 0–9;
- `\w` (`\W`) — буквы, цифры, знак подчёркивания.

Определение. **Перечисление** `[...]` — один любой символ из набора, внутри можно использовать диапазоны: `[a-fA-F0-5]`. **Инвертированное** перечисление: `[^abc]` — любой символ, кроме перечисленных. Внутри `[]` метасимволы означают сами себя: `[.]` — просто точка.

Определение. **Квантификаторы** — сколько раз повторять предыдущий символ или группу:

- `*` — ноль или более повторений; `+` — одно или более; `?` — ноль или одно;
- `{n}` — ровно n раз; `{n,m}` — от n до m раз; `{n,}` — от n до ∞ раз.

Замечание. Формы `{,m}` нет: `a{,5}` — это просто текст, для «от 0 до 5» пишем `a{0,5}`.

Определение. **Типы квантификации**:

- **жадная** (`+`, `*`, `?`, `{}`) — берёт самую длинную подстроку;
- **ленивая** (после квантификатора ставим `?`: `+?`, `*?`, `??`) — берёт самую короткую.

Например, на строке `<b>x</b>` шаблон `<.+>` захватит всю строку, а `<.+?>` — только `<b>`.

7.6 Группировки

Определение. **Группировки** ставятся в круглые скобки:

- `(...)` — **группа с захватом**: запоминает подстроку (достаём по индексу);
- `(?:...)` — **группа без захвата**: ничего не запоминает, просто делит шаблон на логические блоки;
- `|` — «или», самый низкий приоритет, поэтому обычно берётся в группу.

Пример: `(?:red|green) light` (без скобок `red|green light` значит «red» или «green light»). Группы нумеруются по **открывающей** скобке слева направо, `res[0]` — всё совпадение.

Определение. **Просмотр вперёд и назад** (lookaround) — проверка контекста, который не входит в результат:

- `(?=string)` — дальше должно быть `string`;
- `(?!string)` — дальше `string` быть не должно;
- `(?<=string)`, `(?<!string)` — то же, но назад (в `std::regex` не поддерживается).

Пример. Шаблон `best(?= match)` на строке «best match» даёт `res[0]` = «best»: «match» проверили, но в результат не взяли.

Определение. **Обратная ссылка** (backreference) `\num` — ссылка на группу захвата с номером `num`: ровно тот же текст, который она захватила.

Пример. Шаблон `(red|green) apple is \1` подойдёт к «red apple is red» и «green apple is green», но не к «green apple is red».

Задача	Шаблон	Замечание
Файл-картинка	`.*\.(jpg|png|gif)`	точка экранирована
Дата ДД-ММ-ГГГГ	`\d{2}-\d{2}-\d{4}`	то же, что `[0-9]{2}-`…
HEX-цвет	`#[A-Fa-f0-9]{6}`	не забыть строчные `a-f`

Таблица 3. Практические шаблоны

Частые ошибки.

- путать `match` (вся строка) и `search` (подстрока);
- забывать удваивать `\` в обычных строках C++;
- ждать от `{,n}` смысла «до n»;
- не брать `|` в скобки: `red|green light` $\neq$ `(?:red|green) light`.

7.7 Лексический анализ на практике

Токены образуют регулярный язык $\Rightarrow$ лексический анализатор строится как конечный автомат (обычно через рекурсивные функции).

Требования к лексическому анализатору.

1) время работы $O(n)$, где n — длина входного текста;
2) реализация через конечный автомат;
3) поиск ключевых слов по бору, а не перебором списка.

Типы токенов:

1) ключевые слова (key words);
2) операторы (operator);
3) литералы (literal);
4) идентификаторы, т. е. имена переменных (id);
5) разделители (delimiter, опционально);
6) неизвестные символы (unknown).

7.8 Форма Бэкуса–Наура (BNF)

Обычная запись	BNF
$\to$	$::=$
S	$\langle S \rangle$
$S \to A \mid AB$	$\langle S \rangle ::= A[B]$
$S \to \alpha B',\ B' \to \beta B' \mid \varepsilon$	$\langle S \rangle ::= \alpha\{\beta\}$

Таблица 4. Обычная запись правил и BNF

Квадратные скобки — **необязательная часть**, фигурные — **повторение** (ноль и более раз). Обычно ε в BNF не используют.

8 Синтаксический анализ

8.1 Пример и классификация алгоритмов

```
if (a > b)
    a = a - b;
```

⇓ лексический анализатор

`if` `(` `a` `>` `b` `)` `a` `=` `a` `-` `b` `;`

⇓ синтаксический анализатор

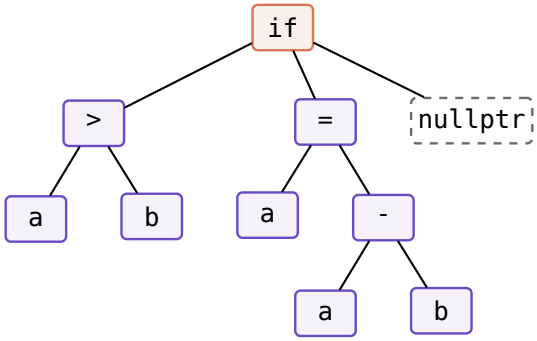


Рис. 7. Синтаксическое дерево: условие, тело и пустая ветка `else`

Универсальные алгоритмы:

1) КЯК (Кока–Янгера–Касами): $O(n^3)$, n — длина ввода;
2) алгоритм Эрли: $O(n^3)$.

Неуниверсальные алгоритмы:

1) LL-анализ (рекурсивный спуск);
2) LR-анализ.

8.2 Рекурсивный спуск

Пример.

$$\langle S \rangle ::= \langle E \rangle \, \$$$
$$\langle E \rangle ::= \langle T \rangle \, \{+ \langle T \rangle\}$$
$$\langle T \rangle ::= \langle F \rangle \, \{* \langle F \rangle\}$$
$$\langle F \rangle ::= \langle Num \rangle \mid (\langle E \rangle)$$

Разбираем строку: $5 + 7 * (4 + 3 * 5)\$$.

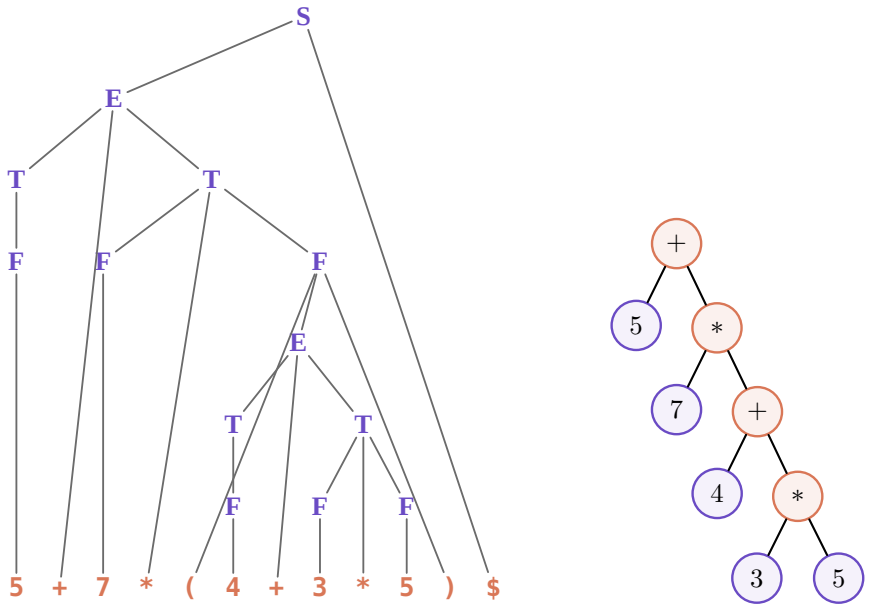


Рис. 8. Дерево разбора строки $5 + 7 * (4 + 3 * 5)\$$ (слева) и получаемое синтаксическое дерево (справа)

Как писать. t — текущий токен, `gt()` — взять следующий токен.

Проверка:

```
void S() {
    E();
    if (t != '$') {
        throw false;
    }
    gt();
}

void E() {
    T();
    while (t == '+') {
        gt();
        T();
    }
}

void T() {
    F();
    while (t == '*') {
        gt();
        F();
    }
```

```
}

void F() {
    if (t == '(') {
        gt();
        E();
        if (t != ')') {
            throw false;
        }
        gt();
        return;
    }
    if (t.type == "num") {
        gt();
        return;
    }
    throw false;
}
```

Построение:

```
Node* E() {
    // операция + левоассоциативна
    Node* left = T();
    while (t == '+') {
        gt();
        Node* tmp = new Node('+');
        tmp->left = left;
        tmp->right = T();
        left = tmp;
    }
    return left;
}
```

Для присваивания (правоассоциативный оператор):

$$\langle A \rangle ::= \langle I \rangle \mid \langle I \rangle = \langle A \rangle$$

```
Node* A() {
    Node* left = I();
    if (t == '=') {
        gt();
        Node* tmp = new Node('=');
        tmp->left = left;
        tmp->right = A();
        left = tmp;
    }
    return left;
}
```

8.3 Применимость рекурсивного спуска

Определение. $\text{First}(\alpha)$ — множество терминалов, с которых может начинаться α, $\alpha \in (T \cup N)^*$.

Пример.

$$\begin{array}{l} S \to aA \mid BS \\ A \to bB \mid cB \mid \varepsilon \\ B \to aA \mid e \end{array} \qquad \text{First}(ABb) = \{b, c, a, e\}, \quad \text{First}(\varepsilon) = \emptyset.$$

Определение. $\text{Follow}(A)$ — множество терминалов, которые могут следовать за A, $A \in N$.

Пример.

$$\begin{array}{l} S \to AaS \mid Ab \\ A \to aA \mid \varepsilon \end{array} \qquad \text{Follow}(A) = \{a, b\}.$$

First для нетерминалов $\text{First}(A)$.

1) $A \to a\alpha \Rightarrow$ добавляем a;
2) $A \to B\alpha \Rightarrow$ добавляем $\text{First}(B)$. Если $B \Rightarrow^* \varepsilon$, добавляем $\text{First}(\alpha)$.

First для цепочек $\text{First}(\alpha)$.

1) $\alpha = a\beta \Rightarrow \{a\}$;
2) $\alpha = A\beta \Rightarrow \text{First}(A)$. Если $A \Rightarrow^* \varepsilon$, то $\text{First}(A) \cup \text{First}(\beta)$.

$\text{Follow}(A)$.

1) $B \to \xi A\alpha \Rightarrow$ добавляем $\text{First}(\alpha)$;
2) $B \to \xi A \Rightarrow$ добавляем $\text{Follow}(B)$;
3) $B \to \xi A\alpha$ и $\alpha \Rightarrow^* \varepsilon \Rightarrow$ добавляем $\text{Follow}(B)$.

Пример.

$$\begin{array}{lll} S \to aA \mid BDc & \text{First}(B) = \emptyset, & \text{Follow}(S) = \emptyset, \\ A \to BAa \mid aB \mid b & \text{First}(D) = \{b\}, & \text{Follow}(A) = \{a\}, \\ B \to \varepsilon & \text{First}(S) = \{a, b, c\}, & \text{Follow}(B) = \{a, b, c\}, \\ D \to B \mid b & \text{First}(A) = \{a, b\}, & \text{Follow}(D) = \{c\}. \end{array}$$

Критерии применимости рекурсивного спуска. Раскрываем A: $\alpha A\beta$ (нужно раскрытие единственным образом).

1) $A \to \alpha \mid \beta$: $\quad \text{First}(\alpha) \cap \text{First}(\beta) = \emptyset$;
2) $A \Rightarrow^* \varepsilon$ и $A \to \alpha_1 \mid \cdots \mid \alpha_n$: $\quad \exists!\, i : \ \alpha_i \Rightarrow^* \varepsilon$;
3) $A \Rightarrow^* \varepsilon$: $\quad \text{First}(A) \cap \text{Follow}(A) = \emptyset$.